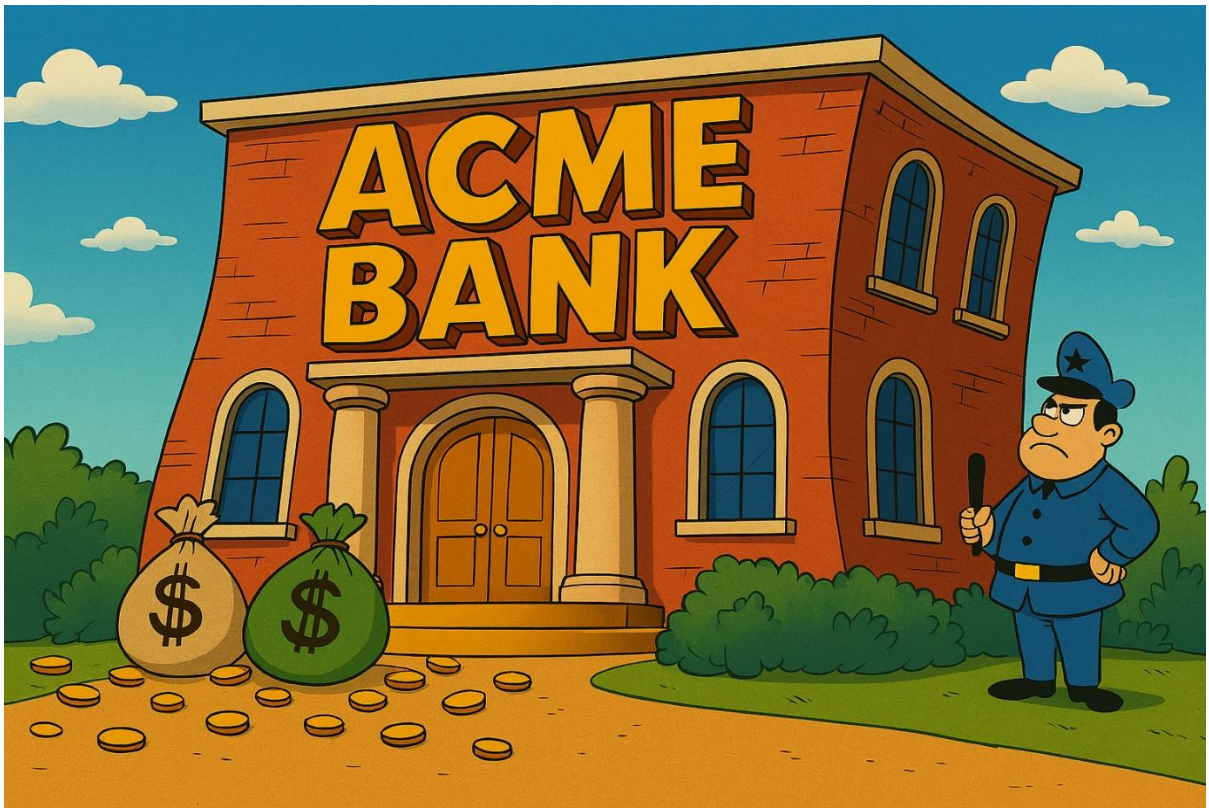
Consultant Assessment Brief
Java Group Project:

Banking Application Programme



1. Overview

Your team has been approached by **Acme Bank** to develop a **Java-based banking system** exclusively for use by **bank tellers**. This system will enable tellers to authenticate customers, manage different types of accounts, and perform various banking operations through a command-line interface (CLI). As consultants, you are expected to maintain professionalism, deliver a technically robust solution, and fulfil the client's requirements. The project team should consist of at least three members.

Specifically, the banking system must provide secure authentication so tellers can confirm customer identities before any transactions take place. It should support multiple account types such as checking, savings, and business accounts and empower tellers to handle essential operations like deposits, withdrawals, balance inquiries, and fund transfers. The CLI should be both user-friendly and reliable, allowing tellers to navigate menus, enter commands, and access transaction histories efficiently.

In fulfilling your professional responsibilities, it is crucial that the software meets industry standards for security and reliability. Code should be thoroughly documented. Team members are encouraged to collaborate closely, taking on roles such as lead developer, tester/QA specialist, or documentation manager, to ensure comprehensive attention to design, development, testing, and deployment. Your ultimate objective is to deliver a scalable, maintainable solution that supports bank tellers' daily activities while fully complying with Acme Bank's specifications.

2. What You Need to Build. What You Need to Build

Your task is to create a **fully featured Java console-based banking system** that supports customer authentication, multiple account types, transaction processing, automated account number and sort code management, and robust error handling. Your application should demonstrate strong object-oriented design, clear separation of concerns, and resilience against invalid input.

This expanded section provides significantly more detail to guide your system design.

Customer Authentication

Customer authentication forms the entry point for all teller interactions. Your system must:

- Enable tellers to locate and identify a customer using a unique identifier such as a **Customer ID**, **National Insurance number**, or another chosen identifier.
- Prevent access to account operations until a customer has been authenticated.
- Handle cases where:
    - A customer does not exist.
    - Incorrect or malformed input is provided.
    - Multiple customers share similar attributes (your system must resolve or disambiguate).
- Allow tellers to switch between customers without restarting the program.

The authentication process does *not* need real-world encryption or secure login, but it must be **consistent, reliable, and resistant to system crashes**.

Account Numbers and Sort Codes (New Requirements)

Every account created in the system must be assigned an **8-digit account number** and the correct sort code automatically. These identifiers must never be entered manually by the teller.

Account Number Requirements

Your system must:

- Generate a **random or sequential 8-digit number** (e.g., `12345678`).
- Guarantee uniqueness by:
    - Checking against all existing accounts before allocating, or
    - Using a structured algorithm (e.g., incremental counters, UUID truncation, or a generator class).

- Handle collisions (e.g., if a random number happens to match an existing account).
- Store the account number in a stable format for display, lookups, and reporting.
- Prevent creation of any account without a valid 8-digit number.

Recommended design: Implement a dedicated `AccountNumberGenerator` class responsible solely for producing unique 8-digit numbers.

Sort Code Rules

Sort codes depend strictly on account type and must be hard-coded into the system:

- **Personal Account:** `60-60-60`
- **ISA Account:** `60-60-70`
- **Business/Company Account:** `60-70-70`

Your system must automatically:

- Assign the correct sort code at the moment of account creation.
- Prevent modification of sort codes after the account is created.
- Display sort codes consistently in teller interfaces.

Account Types and Rules

Each account type must be implemented as a specialised class with its own behaviour, restrictions, and validation logic.

Personal Account

Your implementation must:

- Validate the required two forms of ID:
- Photo ID (passport, driving licence)
- Address ID (utility bill, council tax letter)
- Enforce a minimum **£1 opening balance**, rejecting attempts to open with less.
- Support **multiple accounts per customer**.
- Handle **overdraft scenarios**, including calculating charges if applicable.
- Allow tellers to set up and view **Direct Debits** and **Standing Orders**.
- Track all transactions, displaying clear running balances.

ISA Account

ISA accounts require careful rule handling. Your system must:

- Allow only **one ISA per customer** and enforce this rule strictly.
- Calculate annual interest at **2.75% APR** based on:
- The **average annual balance**, or
- A simplified equivalent (if justified in your assumptions document).
- Prevent creation of direct debits/standing orders unless justified by your assumptions.
- Handle attempts to deposit or withdraw in cases where ISA rules prohibit certain actions (you may simplify with assumptions).

Business Account

Your business account logic must:

- Require verification of an **eligible business type**.
- Reject excluded types (Enterprise, PLC, Charity, Public Sector) with clear messages.
- Apply the **annual fee of £120** automatically.
- Handle optional features:
- Cheque book issuance
- Overdraft facility
- International trading functionality (simplified)
- Loan request handling (basic simulation)
- Allow only **one business account per customer**.

The business account class must also track whether the account has received a cheque book and whether overdraft/loan facilities are active.

Required Banking Operations

Your application must support a comprehensive set of operations for each customer and account type.

Account Creation

- Automatically assign sort code and account number.
- Validate eligibility for the chosen account type.
- At creation, record:
    - Customer ID
    - Opening balance
    - Assigned sort code
    - Generated account number

    - Account type metadata (fees, constraints, benefits)

Account Management

Your system must allow tellers to:

- View full account details including:
- Account number
- Sort code
- Current balance
- Account type and rules
- Overdraft/fee/interest information
- View all accounts belonging to the currently authenticated customer.

Transaction Handling

Your system must:

- Allow deposits and withdrawals.
- Prevent invalid withdrawals (e.g., insufficient funds unless overdraft applies).
- Track transaction history internally (full history optional but recommended).
- Warn the teller when transactions may trigger fees or rule violations.

ISA Interest Calculation

The teller should be able to trigger or display:

- Annual interest based on the account's balance history.
- Clear breakdown of how interest is calculated.

Business Account Cheque Book Issuance

The teller must be able to:

- Request a cheque book for an eligible business account.
- Prevent duplicates if one has already been issued.

System Behaviour Requirements

Your Java program must demonstrate financial-system-level robustness in its behaviour.

Menu System

- Provide a clear, intuitive hierarchy of options.
- Provide numbered or keyword-based navigation.

- Prevent navigation errors (e.g., invalid menu choices).

Input Validation

Your system must:

- Reject malformed numbers, negative deposits, invalid characters, etc.
- Continue running regardless of invalid input.
- Never crash due to user error.

Help System

You must implement **context-aware help**, meaning:

- Help displays information relevant to the current action or menu.
- Help is accessible from any point in the application.
- Help describes account rules, transaction restrictions, and system behaviour.

Assumptions Document

Any rule you simplify must be:

- Listed clearly in a separate assumptions file.
- Justified (e.g., simplifying ISA tax-year handling).

Git Collaboration Requirements

Your group must:

- Use a shared GitHub repository.
- Commit regularly with meaningful messages.
- Avoid working only on the main branch.
- Document member names in the README.

3. Technical Expectations (Java)

Technical Expectations (Java) Use strong object-oriented design principles.

Class Design Expectations

Your system architecture should follow a clear object-oriented structure, using **PascalCase** for all class names to maintain readability and consistency with industry conventions. Rather than relying on a fixed set of predefined classes, the design should naturally evolve from the system's responsibilities. This includes separating concerns such as customer management, account logic, identifier generation, authentication, data handling, logging, and user interaction.

Classes should be organised so that:

- Each class has a single, clear responsibility.
- Inheritance and polymorphism capture shared behaviour between account types.
- Supporting operations—such as generating account numbers, storing and loading data, validating input, logging events, and controlling menus—are handled by dedicated components.
- The design is scalable, allowing new account types or features to be added with minimal rework.

The structure should reflect professional banking system architecture, with expressive PascalCase names that clearly communicate their role within the application.

Persistence

Data should be stored in a persistent format such as JSON, CSV, or serialized objects so that customer and account information remains available between program executions. Your system should include a mechanism for loading and saving all banking data using clearly defined data access classes.

Logging

Your application should maintain a logging mechanism to record key teller actions, account events, transactions, and system warnings. Logs should be written to a file and timestamped.

Error Handling

Use Java exception handling to ensure the program never terminates unexpectedly.

4. Group Work Requirements

- **Use a shared GitHub repo**
  Setting up a shared repository on GitHub ensures that everyone on the team has access to the project's codebase, resources, and documentation. This central platform enables seamless collaboration and version control, making it easier to track changes and contributions.

- **Make regular, clear commits**
  Committing code often, and with descriptive messages, helps other team members stay updated on progress and understand what each change accomplishes. Clear commit history can also simplify debugging and reviewing past decisions.

- **List all team members in the README**
  Including the names and contact information of all contributors in the README provides transparency and recognition. It allows newcomers and collaborators to know who is involved and where to direct questions.

- **Coordinate development through effective communication**
  Establishing consistent channels for communication, such as chat groups or scheduled meetings, minimizes misunderstandings and keeps everyone informed about goals, responsibilities, and deadlines. Open dialogue strengthens teamwork and project success.

5. What You Must Submit

Your submission must include:

- **Java source code**
  This refers to the complete set of files and code written in Java that make up the application's functionality.

- **README listing team members**
  The README file should include a roster of all contributors involved in developing the project, ensuring credit and accountability.

- **Assumptions summary**
  This document outlines the key assumptions made during development, clarifying any constraints or expected conditions relevant to the system.

- **User guide**
  A manual intended for end users, explaining how to install, configure, and use the software step by step.

- **Test evidence: structured cases and screenshots**
  Comprehensive documentation of tests performed, including detailed test cases and visual proof such as screenshots, to demonstrate that the software works as intended.

- **UML diagrams: Class, Use Case, Sequence (core workflows)**
  Visual representations using Unified Modelling Language to illustrate system architecture, user interactions, and main process flows.

- **Implemented persistence and logging**
  Features in the system that allow data to be saved permanently and events or errors to be recorded for review and troubleshooting.

- **Account number generator and validation**
  Functionality within the application that creates new account numbers and ensures they are valid according to specified rules or formats.

- **Submissions not later than:**
  You must submit your work no later than 5:00 pm (GMT) 21st April 2026. No files should be altered after this deadline. Any files found to be updated after this deadline will be deducted from evaluation, and may affect your final result.

6. How You Will Be Assessed

Your project will be evaluated across several dimensions that reflect professional banking software development standards. Each area below represents a critical component of both technical competence and collaborative engineering practice.

Functionality

The system must implement every feature described in the requirements, including detailed account rules, automated number and sort code generation, transaction handling, persistence, logging, and accurate enforcement of ISA and business constraints. Functionality is assessed not only by what features exist but by how reliably and consistently they behave. Edge cases, rule enforcement, error prevention, and correct financial behaviour all contribute to this area.

Correctness

Correctness focuses on accuracy of operations and internal logic. Your system should:

- Produce correct calculations for balances, interest, overdrafts, and fees.
- Validate user input thoroughly and consistently.
- Maintain accurate state across sessions through persistence.
- Prevent invalid operations such as duplicate ISA accounts, unsupported business types, or malformed account numbers.
- Ensure generated account numbers remain unique.

Correctness also includes compliance with the rules for each account type and reliable execution of the help system.

Software Design

This area evaluates how well your solution follows object-oriented principles and architectural clarity. Strong design includes:

- Meaningful class decomposition and coherent abstraction.
- Proper use of inheritance and polymorphism for account types.
- Dedicated classes for authentication, data management, logging, account number generation, and menu control.
- Loose coupling and high cohesion.
- Clear separation of concerns between UI, logic, data, and utilities.
- Your UML diagrams must accurately reflect the structure of your implemented system.

User Experience

Even though the program is console-based, user experience matters significantly. Your interface should:

- Present clear menus and navigation options.
- Display readable, formatted account information.
- Provide concise and relevant help at any point.
- Prevent confusion by guiding the teller logically through operations.

A strong UX reduces cognitive load and mirrors the reliability expected in real banking systems.

Group Collaboration

Collaboration is evaluated through:

- Commit frequency and quality.
- Evidence of shared responsibility across the team.
- Branching practices and pull request history.
- Clear documentation of team roles, if applicable.
- Peer reviews will require every team member to evaluate both their own contribution and the contributions of their peers. Each candidate should reflect on:
- Their individual workload, responsibilities, and technical input.
- How effectively they collaborated, communicated, and supported the team.
- The quality, readability, and maintainability of the code they produced.
- The strengths and areas for improvement observed in other team members' work.

These reviews should be constructive, honest, and focused on clear evaluation rather than vague encouragement. They must be recorded formally and emailed directly to the tutor for review. This ensures transparency in evaluating individual contribution, team collaboration, and overall project participation. The peer review process is intended to promote accountability, improve team effectiveness, and encourage thoughtful assessment of both personal and collective performance.

Documentation

Your documentation will be assessed for accuracy, clarity, and completeness. This includes:

- A structured README.
- A detailed user guide.
- Comprehensive assumptions document.
- Test documentation showing input, expected output, and results.
- UML diagrams that align with the codebase.

Strong documentation demonstrates professionalism and ensures maintainability.

7. Additional Required Capabilities

The following elements enhance the robustness and professionalism of your banking system and must be fully integrated into the final solution. These requirements elevate the system beyond a simple console app and reflect industry practices.

Logging and Audit Trail

The system should record significant events such as account creation, failed authentication attempts, deposits, withdrawals, fee applications, and system warnings. Logs must:

- Be timestamped.
- Be stored in persistent files.
- Provide clear insight into teller actions and system behaviour.

Logging is essential for traceability, troubleshooting, and compliance.

Data Persistence

Customer and account information must persist across program executions. This requires implementing:

- File-based storage (JSON, CSV, or serialized objects).
- A structured data loading and saving system.
- Validation when loading data to prevent corruption.

This ensures continuity of banking operations and simulates real financial system storage requirements.

Testing Expectations

Your testing suite must demonstrate:

- Coverage of account creation, transaction workflows, and rule enforcement.
- Validation of interest calculations and fee applications.
- Handling of invalid input and attempted rule violations.
- Behaviour at boundary cases (e.g., zero balances, maximum deposits, duplicate account restrictions).

Tests should be documented clearly and demonstrate reliability in real-world-like scenarios.

Identifier Generation Systems

Your system must include dedicated logic for generating and validating:

- Unique 8-digit account numbers.
- Correct sort codes based on account type.

This component must operate reliably under repeated account creation and prevent collisions.

Comprehensive Menu and Help Framework

Your user interface must allow tellers to:

- Navigate through all system actions intuitively.
- Access context-sensitive help at any stage.
- Benefit from consistent and structured menu hierarchy.

This framework should reflect the workflow of a bank teller system, promoting ease of use and reducing training burden.

8. Marking Scheme

The following marking scheme outlines how the consultancy deliverable will be evaluated. Each category reflects expectations consistent with industry-grade software development and client delivery standards.

1. Functional Compliance – 25%

Assesses the extent to which the delivered system meets all specified requirements, including:

- Correct implementation of account rules and constraints.
- Successful automation of account number and sort code generation.
- Transaction handling, interest calculations, and operational workflows.
- Full support for teller-based interaction and customer authentication.

2. Technical Accuracy and Correctness – 20%

Evaluates:

- Accuracy of financial calculations (balances, fees, interest).
- Stability of the system during invalid input or edge cases.
- Correct enforcement of account restrictions (e.g., ISA uniqueness, business rules).
- Data integrity across all operations.

3. Software Architecture and Design Quality – 20%

Assesses the sophistication and clarity of the system design, including:

- Appropriate use of object-oriented design and PascalCase class structures.
- Separation of concerns across the system.
- Effective use of design principles such as inheritance, abstraction, and modularity.
- Scalability and maintainability of the architecture.

4. Documentation and Professional Presentation – 10%

Includes evaluation of:

- The clarity and organisation of submitted documentation.
- Thoroughness of the user guide, assumptions document, and UML diagrams.
- Professionalism and readability of written materials.

- Alignment of documentation with the implemented system.

5. Testing and Quality Assurance – 10%

Assesses the consultant team's testing approach and evidence, including:

- Structured test cases.
- Boundary testing.
- Demonstrated handling of incorrect or unexpected input.
- Evidence of system reliability and robustness.

6. Collaboration, Peer Review, and Version Control – 10%

Measures:

- Quality and consistency of contributions across the team.
- Evidence of effective communication and shared responsibility.
- Version control practices (branching, commit quality, PR reviews).
- Completion and submission of peer reviews detailing individual and team contributions.
- All files must be submitted by the agreed date and time, any files exceeding this date and time will be ignored and may affect the overall marking scheme of the project.

7. Logging, Persistence, and System Infrastructure – 5%

Evaluates:

- Implementation of a functioning logging system.
- Reliability and validity of data persistence mechanisms.
- Structured storage and retrieval of customer/account data.